Attention-Enhanced PPO for UGV Task Dispatching in UAV-Assisted Open-RMF Logistics

Tianyu Lu*, Yihao Zhu†, Qing Yang†
* Department of Computer Science and Engineering
Texas Academy of Mathematics and Science
Denton, United States
† Department of Computer Science and Engineering
University of North Texas
Denton, United States

***Abstract*—Unmanned Aerial Vehicles (UAVs) and Unmanned Ground Vehicles (UGVs) are increasingly deployed for rapid target discovery and ground-level object search. In the present system, UAVs provide target discovery and task-notification support, while the learned reinforcement-learning dispatcher controls three UGV delivery robots through an Open-RMF navigation interface. This paper formulates the UGV dispatching problem as a discrete Markov decision process with a 63-dimensional observation, a four-action robot-selection space, a specified reward function, and a one-second decision interval. We compare PPO-based dispatching with A2C, DQN, greedy scheduling, and round-robin scheduling in an airport-terminal simulation. We then analyze internal PPO variants, including a 63-dimensional MLP policy, a 54-dimensional MLP ablation without estimated start-time guidance, and an attention-enhanced PPO encoder. The results show a trade-off rather than uniform dominance: Attn-PPO achieves the lowest total completion time and queue variance in the tested configuration, while greedy dispatching obtains lower local waiting time and movement distance. These findings position attention as a relation-aware representation improvement inside PPO for the reported single-map, three-UGV Open-RMF dispatching setting.**

***Index Terms*—Reinforcement learning, proximal policy optimization, attention mechanism, multi-robot task allocation, UAV–UGV cooperation, Open-RMF, task dispatching.**

I. Introduction

Multi-robot task dispatching plays an important role in autonomous service systems, logistics automation, inspection, and smart transportation facilities. In large indoor environments such as airport terminals, ground robots may need to respond to tasks discovered by aerial sensing or external monitoring systems under dynamically arriving tasks, spatially distributed waypoints, and changing execution states. The dispatcher must decide which robot should serve each task while considering robot positions, queue lengths, current execution status, task waiting time, and the overall system workload. Poor dispatching decisions may lead to long task waiting times, unbalanced robot workloads, unnecessary movement, and reduced system throughput.

Open-RMF provides a practical middleware framework for coordinating fleets of robots in shared environments. However, designing an efficient dispatching policy in an RMF-based system remains challenging. Rule-based methods such as round-robin assignment or greedy algorithm are simple and interpretable, but they often fail to account for long-term effects such as future queue congestion and makespan increase. Reinforcement learning provides a promising alternative because it can learn dispatching policies from interaction data and optimize long-term cumulative rewards. Algorithms such as PPO and A2C are widely used due to their stability and compatibility with sequential decision processes.

Nevertheless, directly applying standard reinforcement learning methods to multi-robot dispatching has an important limitation: the dispatching state is often represented as a flat numerical vector. In the current system, the observation contains robot states, pending task states, global statistics, and estimated task start times. Although this representation contains useful information, a standard multilayer perceptron (MLP) policy does not explicitly preserve the entity-level structure among robots and tasks. As a result, the policy must implicitly learn robot–task relationships from fixed feature positions, which may reduce sample efficiency and generalization when task configurations change.

To address this problem, this paper presents a PPO-based UGV dispatching framework in a UAV-assisted Open-RMF airport terminal environment. PPO is selected as the main learning algorithm because it provides stable policy-gradient updates and performs well with the compact discrete action space used by the dispatcher. Within the PPO family, we study several representation modes. The standard PPO-MLP-63D variant uses the full 63-dimensional observation as a flat vector. The PPO-MLP-54D ablation removes the 3×3 estimated start-time guidance block. The Attn-PPO variant converts the dispatching observation into structured tokens: one global token, multiple robot tokens, and multiple task tokens. A multi-head self-attention module is then used to model the relationships among these tokens. This organization allows the paper to first compare PPO-based dispatching with non-PPO baselines, and then separately analyze the effect of PPO internal representation choices.

The main contributions of this paper are summarized as follows:

- A discrete MDP formulation is specified for an Open-RMF UGV dispatcher, including a 63-dimensional observation, four-action robot-selection interface, reward

components, episode logic, and dispatch timing.
- A PPO-based dispatcher is evaluated against A2C, DQN, greedy scheduling, and round-robin scheduling under the same UAV-assisted task-notification and RMF-compatible execution pipeline.
- Multiple PPO representation modes are analyzed, including PPO-MLP-63D, PPO-MLP-54D, and Attn-PPO, to separate algorithm-level performance from representation-level effects in the RMF dispatcher and to state the observed trade-offs explicitly.

II. Related Work

A. Multi-Robot Task Allocation

Multi-robot task allocation (MRTA) assigns tasks to robots while considering capability, travel cost, workload balance, and mission completion time. Classical methods include centralized optimization, auction-based allocation, market-based mechanisms, greedy assignment, and rule-based scheduling [1]. Recent surveys further organize MRTA methods into optimization-based, market-based, and behavior-based families, and emphasize that exact optimization methods can become difficult to apply in dynamic or large-scale settings [2]. Complexity and approximation analyses also show why approximate and learning-based methods are attractive when assignment constraints, robot budgets, and task requirements grow [3]. Recent systems also combine task allocation with learning or hybrid coordination, including end-to-end DRL for modular autonomous mobile-system allocation [9], graph-neural multi-agent task allocation for heterogeneous autonomous vehicles [10], and centralized hybrid optimization for heterogeneous warehouse robots [11]. These approaches are often efficient and interpretable, but many of them either optimize local or handcrafted criteria or study broader heterogeneous-allocation settings than the RMF UGV dispatch interface used here. In dynamic logistics, each assignment affects both immediate travel cost and future fleet workload, so a dispatcher should account for delayed consequences.

B. Reinforcement Learning for Dispatching

Reinforcement learning is suitable for dispatching because each decision affects future queue length, robot idleness, task waiting time, and final makespan. PPO provides a stable policy-gradient framework by optimizing a clipped surrogate objective [4]. PPO-based MRTA has also been studied in air-ground and multi-agent forms, including PPO scheduling for RMF-compatible UAV–UGV collaboration [5], MAPPO-based dynamic coalition formation for tasks that require cooperative groups of robots [6], and MAPPO-based spectrum and power reallocation for dynamically reconfigurable UAV–UGV clusters [8]. Hierarchical and bi-level MRTA methods combine assignment with lower-level policy learning under uncertainty or temporal-logic constraints [7]. Compared with these multi-agent, communication-resource, or hierarchical formulations, the present paper studies a single centralized dispatcher that selects a UGV for each visible task request through Open-RMF; this is a narrower formulation, but it matches the deployed RMF task-dispatch interface used in the airport-terminal simulation. In this work, Attn-PPO, PPO, and A2C are implemented using Stable-Baselines3 [19]. The key difference between PPO and Attn-PPO is not the optimizer, but the state representation consumed by the actor–critic policy. We also distinguish PPO-MLP-63D and PPO-MLP-54D to study the effect of estimated-time guidance features.

C. Attention for Structured Decision Making

Attention mechanisms provide a principled way to model interactions among entities. The Transformer architecture demonstrated the effectiveness of self-attention for structured token interaction [12]. Attention-based and graph-based DRL have recently been used for UAV–UGV collaborative route planning and heterogeneous allocation, including heuristic-assisted route planning, attention-aware route construction, energy-constrained cooperative routing with transformer encoders, and MAGNNET's GNN-based decentralized allocation [10], [13]–[15]. Related UAV–UGV planning work also studies fuel-constrained cooperative routing and risk-aware recharging rendezvous under energy uncertainty [16], [17], while heterogeneous-agent MARL work studies centralized feedback and reward shaping for decentralized UAV–UGV path planning under energy constraints [18]. These works primarily address cooperative routing, recharging rendezvous, multi-agent route construction, or decentralized allocation. By contrast, the present paper focuses on Open-RMF task dispatching: UAVs provide sensing and task notification, while the RL action selects among UGV delivery robots. In this setting, the suitability of a UGV for a task depends on robot position, queue state, task location, task waiting time, and fleet-level congestion. Attn-PPO uses this property to improve state representation before PPO action selection. In the present study, attention is interpreted primarily as a configuration-specific relational inductive bias: it can stabilize dispatch decisions in the tested setting by explicitly modeling robot–task–global interactions instead of relying only on fixed scalar feature positions.

III. Proposed Method

A. MDP Formulation and Dispatch Interface

The dispatching problem is formulated as a discrete Markov decision process. The controllable agents in the current RL loop are three UGVs, denoted by $\mathcal{R} = \{r_0, r_1, r_2\}$. UAVs are not selected by the policy in this version; they are modeled as a sensing and task notification layer that creates target-search requests for the UGV dispatcher.

At decision step t, the policy observes $o_t \in \mathbb{R}^{63}$. The observation contains a robot block $s_t^R \in \mathbb{R}^{3\times 7}$, a task block $s_t^T \in \mathbb{R}^{8\times 3}$, a global block $s_t^G \in \mathbb{R}^9$, and an estimated start-time block $s_t^E \in \mathbb{R}^{3\times 3}$. For each UGV, the robot block stores normalized position, idle flag, queue length, queue anchor position, and remaining execution time. The task block stores normalized task position and waiting time for up to eight visible pending tasks. The global block stores normalized

simulation time, completion ratio, pending-task ratio, idle-robot ratio, total queue length, active-task ratio, queue-length variance, maximum queue length, and queue gap. The estimated start-time block estimates how soon each UGV could start each of the first three visible pending tasks.

The action space is

$$\mathcal{A} = \{0, 1, 2, 3\}, \tag{1}$$

where actions 0, 1, and 2 select deliveryRobot_0, deliveryRobot_1, and deliveryRobot_2, respectively, and action 3 is a no-op or wait action. The learned policy therefore selects a robot or waits; it does not directly select a robot–task pair. When a valid UGV action is chosen and the pending queue is nonempty, the environment assigns to that UGV the closest pending task measured from the robot's current queue anchor. If the selected UGV is already executing a task, the new task is appended to its queue, and the queued tasks are reordered by distance from the current anchor.

The one-step reward is a clipped weighted sum of assignment, waiting, travel, congestion, completion, and makespan terms:

$$\begin{aligned} r_t = \mathrm{clip}_{[-20,20]}\big(&c_0 + c_v \mathbf{1}_{\mathrm{valid}} + c_i \mathbf{1}_{\mathrm{invalid}} + c_w \mathbf{1}_{\mathrm{wait,pending}} \\ &+ c_p |\mathcal{P}_t| + c_d d_t + c_s \tau_t + c_q q_t^2 + c_m \Delta M_t \\ &+ c_c n_t^{\mathrm{done}} + c_a \mathbf{1}_{\mathrm{all}} + c_b \mathbf{1}_{\mathrm{batch}}\big), \end{aligned} \tag{2}$$

where $\mathcal{P}_t$ is the pending-task set, d_t is the assignment distance, τ_t is the selected robot's queued execution time before the new task can start, q_t is the selected robot's queue length after assignment, and ΔM_t is the reduction in the estimated fleet makespan caused by the assignment. The coefficients are $c_0 = -0.01$, $c_v = 5.0$, $c_i = -5.0$, $c_w = -5.0$, $c_p = -0.05$, $c_d = -0.02$, $c_s = -0.01$, $c_q = -0.2$, $c_m = 0.2$, $c_c = 5.0$, $c_a = 20.0$, and $c_b = 2.0$. These weights penalize idle waiting when tasks are pending, excessive travel, queue congestion, and long pending queues, while rewarding valid assignments, task completions, all-task completion, and assignments that reduce the estimated fleet makespan.

Each dispatch decision advances the simulator by 1 s. In the mock training environment, tasks are sampled by shuffling a fixed airport waypoint pool and releasing one task every 20 s; at most eight pending tasks are exposed to the policy. An episode ends when all released tasks are completed and all UGV queues are empty, or is truncated after 1000 decision steps. At reset, the three UGVs return to fixed initial positions, queues and pending tasks are cleared, and the task order is reshuffled.

B. Baseline PPO Policy

The baseline PPO scheduler receives the complete 63-dimensional observation as a flat vector and encodes it with a multilayer perceptron shared by the policy and value networks. PPO is trained with clipped policy-gradient updates, generalized advantage estimation, and entropy regularization. The 63-dimensional observation contains robot states, task states, global queue statistics, and a 3×3 estimated start-time guidance block. The guidance block is indexed by the first three visible pending tasks and the three UGVs, and each entry estimates how soon a robot could begin serving the corresponding task.

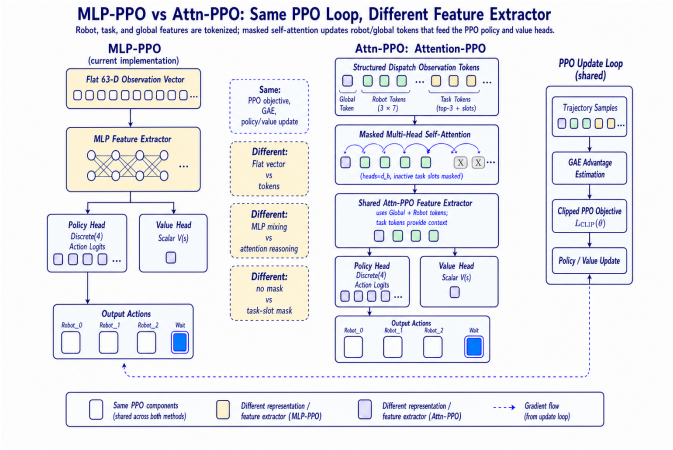


Fig. 1. MLP-PPO and Attn-PPO with different feature extractors.

To evaluate whether the MLP policy depends on these handcrafted features, we also consider a PPO-MLP-54D ablation in which the 3×3 estimated start-time guidance block is removed. This ablation keeps the same action space and PPO objective but forces the MLP to infer robot–task compatibility only from robot, task, and global scalar features. The observed increase in run-to-run variance under this ablation motivates the attention-based representation described below.

C. Attention-Enhanced PPO Architecture

To introduce an inductive bias that matches the dispatching problem, we replace the flat feature extractor of PPO with a structured attention encoder, as shown in Fig. 1. The PPO optimizer, reward function, action space, and dispatching interface are kept unchanged. Therefore, the main difference between plain PPO and Attn-PPO lies in the representation learned before the actor and critic heads.

The attention block itself is a standard masked multi-head self-attention module rather than a new attention mechanism. The contribution in this paper is its integration into an RMF-compatible centralized UGV dispatcher and the specific observation structuring that turns global state, robot state, task state, and estimated start-time guidance into tokens consumed by the PPO actor–critic.

The observation $o_t \in \mathbb{R}^{63}$ is partitioned into three robot feature vectors, eight task feature vectors, global scheduling statistics, and estimated start-time features. The estimated start times are associated with the first three task slots and the three robots. For task slots without an estimated start-time entry, the corresponding pairwise feature is set to zero. Thus, each task token is formed from both task-level information and robot-dependent start-time context.

Let $f_{t,i}^R \in \mathbb{R}^7$, $f_{t,j}^T \in \mathbb{R}^3$, $e_{t,j} \in \mathbb{R}^3$, and $f_t^G \in \mathbb{R}^9$ denote the raw robot, task, estimated start-time, and global features, respectively. The task feature used by the encoder is

$$\tilde{f}_{t,j}^T = [f_{t,j}^T, e_{t,j}] \in \mathbb{R}^6. \tag{3}$$

Robot, task, and global features are then mapped into a common 64-dimensional token space:

$$x_{t,i}^R = W_R f_{t,i}^R + b_R, \tag{4}$$
$$x_{t,j}^T = W_T \tilde{f}_{t,j}^T + b_T, \tag{5}$$
$$x_t^G = W_G f_t^G + b_G. \tag{6}$$

The resulting token matrix is

$$X_t = [x_t^G, x_{t,1}^R, \ldots, x_{t,3}^R, x_{t,1}^T, \ldots, x_{t,8}^T] \in \mathbb{R}^{12\times 64}. \tag{7}$$

D. Masked Self-Attention for Dispatch Relations

The attention encoder allows each valid token to aggregate information from the other tokens before PPO makes a dispatching decision. For a token matrix X, the query, key, and value matrices are

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V. \tag{8}$$

With a padding mask M, the scaled dot-product attention is computed as

$$\text{attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V, \tag{9}$$

where masked inactive task slots receive a large negative value in M. Equivalently, their attention weights are forced to zero:

$$\alpha_{ij} = 0, \quad \text{if task slot } j \text{ is inactive.} \tag{10}$$

This prevents padded task entries from affecting the policy when fewer than eight tasks are pending.

Multi-head attention applies several attention operations in parallel:

$$\text{head}_h = \text{attention}(XW_Q^h, XW_K^h, XW_V^h), \tag{11}$$

$$\text{MHA}(X) = \text{Concat}(\text{head}_1, \ldots, \text{head}_H)W_O. \tag{12}$$

In the implementation, $H = 4$ heads and the token embedding dimension is 64. The output is combined with the input through a residual connection and layer normalization:

$$Z_t = \text{LayerNorm}(X_t + \text{MHA}(X_t)). \tag{13}$$

This structure enables a robot token to attend to pending tasks, other robots, and global congestion information. Consequently, the learned representation can capture relations such as robot–task compatibility, queue pressure, robot idleness, and workload balancing.

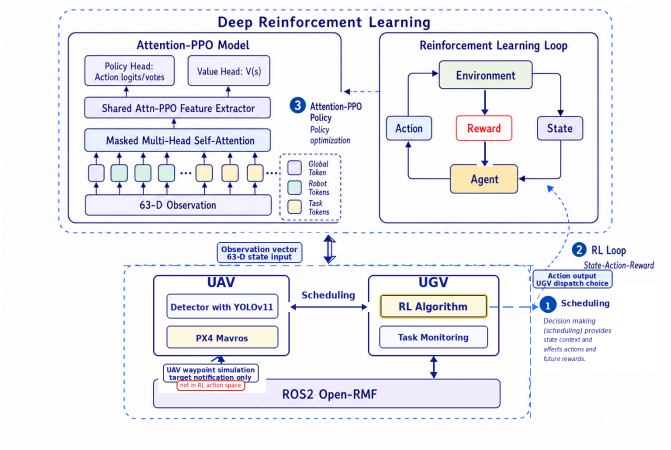


Fig. 2. UAV-assisted UGV dispatching architecture in the Open-RMF airport-terminal simulation. The RL policy dispatches UGVs; UAV simulation supports target discovery and task notification.

E. Coupling the Attention Encoder with PPO

The attention module is used as the feature extractor before PPO's actor and critic. After self-attention, only the updated global token and the three updated robot tokens are pooled:

$$\bar{z}_t = \text{vec}([z_t^G, z_{t,1}^R, z_{t,2}^R, z_{t,3}^R]) \in \mathbb{R}^{256}. \tag{14}$$

The pooled vector is projected into the feature vector used by PPO:

$$h_t = \text{ReLU}(W_p \bar{z}_t + b_p), \quad h_t \in \mathbb{R}^{128}. \tag{15}$$

The policy head and value head are then computed from the same attention-enhanced feature:

$$\pi_\theta(a_t|o_t) = \text{softmax}(g_\pi(h_t)), \tag{16}$$
$$V_\phi(o_t) = g_V(h_t). \tag{17}$$

Thus, task tokens do not directly enter the final PPO heads. Instead, they provide context through self-attention and influence the updated global and robot tokens that are passed to PPO.

PPO maximizes the clipped surrogate objective

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right], \tag{18}$$

where

$$r_t(\theta) = \frac{\pi_\theta(a_t|o_t)}{\pi_{\theta_{\text{old}}}(a_t|o_t)}. \tag{19}$$

The attention encoder changes the state representation, while PPO constrains the policy update to avoid overly large changes. The combined Attn-PPO method therefore uses standard PPO optimization with a relation-aware dispatch representation.

IV. Experimental Setup

A. Simulation Environment

The experiments are conducted in an RMF airport terminal simulation built with ROS 2, Gazebo, and Open-RMF. The

launch pipeline starts the airport terminal scene, task monitor, custom task publisher, RL dispatcher, target object spawning, UAV waypoint simulation, and data recorder. The tested configuration contains three dispatchable UGVs. The UAV component is used to simulate target discovery and waypoint-level task notification; UAV motion is not part of the RL action space in this paper.

Task locations are drawn from a fixed set of airport-terminal waypoints distributed across junctions and corridor nodes. In the mock training environment, the task order is randomly shuffled at reset and one task is released every 20 s. The policy observes at most eight visible pending tasks at a time; additional task arrivals remain outside the visible queue until released. Each run uses the same evaluation recorder and produces a summary containing total experiment time, completed tasks, completion rate, average waiting time, total movement distance, and per-robot task counts. Because all reported results use the same map, three-UGV fleet, and task-generation process, the conclusions should be interpreted as evidence for this airport-terminal configuration rather than as a claim of general performance across arbitrary maps, fleet sizes, or arrival rates.

B. Compared Algorithms

The current evaluation includes:

- PPO family: the proposed PPO-based dispatcher, including MLP and attention-enhanced representation modes.
- PPO-MLP-63D: the default PPO mode with an MLP policy and the full 63-dimensional observation.
- PPO-MLP-54D: an internal PPO ablation that removes the 3×3 estimated start-time guidance block.
- Attn-PPO: an internal PPO mode with a masked self-attention feature extractor.
- A2C: advantage actor–critic with an MLP policy.
- DQN: discrete-action deep Q-network baseline.
- Greedy: nearest-task assignment based on immediate spatial cost.
- Round-robin: cyclic assignment among robots.

C. Training and Hyperparameters

All learning-based methods are implemented with Stable-Baselines3 and trained for 10^6 environment steps unless early stopping is triggered by the validation callback. During training, the current policy is evaluated every 5000 environment steps over five mock-environment episodes, and the final SB3 policy evaluation uses ten episodes. The RMF results in Table II are reported as mean $\pm$ standard deviation over the repeated RMF evaluation runs available for each method. Thus, the variance in Table II reflects evaluation rollouts under shuffled task order and corresponding trained checkpoints; it should not be interpreted as a complete multi-seed retraining study. A broader multi-seed evaluation is left for future work. In particular, the current data do not provide training-return curves with confidence bands over independent random seeds, so they cannot separate policy-learning stability from RMF rollout variability.

D. Inference Timing

For real-time RMF deployment, the relevant question is whether the policy forward pass is small relative to the dispatch interval. The current implementation runs inference on CPU and dispatch decisions are taken once per simulated second. A single-thread PyTorch microbenchmark on the development CPU gives approximately 0.03 ms per decision for a PPO-MLP-63D-sized forward pass and 0.17 ms per decision for a 12-token, 64-dimensional, four-head attention forward pass. These values are approximate network-forward timings rather than full ROS callback timings, but they indicate that the attention overhead is small relative to the 1 s dispatch period and to RMF navigation execution time. If the visible task queue or robot count is increased, however, the attention cost grows with the square of the token count, so hardware-specific latency should be remeasured for larger deployments.

E. Evaluation Metrics

Four metrics are used in the updated comparison. Total completion time measures the final mission duration. Average waiting time measures the mean delay before task service. Average movement distance measures robot travel cost. Queue variance measures workload imbalance among robots; lower values indicate more balanced robot utilization.

V. Results and Discussion

Table II reports the comparative results generated by the analysis pipeline, while Table III summarizes the available variance evidence for internal PPO representation modes. The results should be read as a multi-objective comparison in the tested airport-terminal setup rather than as proof that one algorithm dominates all dispatching metrics. They also should not be interpreted as evidence of scaling behavior on different maps, larger robot fleets, different task-arrival rates, or larger visible task queues.

A. Overall Dispatching Performance

Table II shows that Attn-PPO obtains the lowest total completion time and the lowest queue variance among the evaluated methods. This suggests that relation-aware PPO is well matched to the current RMF dispatching formulation when the objective emphasizes fleet-level completion and workload balance. However, Attn-PPO is not best on every local metric: greedy dispatching obtains the lowest average waiting time and slightly shorter movement distance because it directly favors immediate nearest-task assignment. The practical interpretation is therefore a trade-off: Attn-PPO accepts higher waiting time and movement distance than greedy in exchange for much lower queue variance and shorter mission completion time. PPO-MLP-63D is treated as the default strong PPO baseline because the full 63-dimensional observation provides estimated-time guidance, while Attn-PPO is analyzed as a representation mode that can improve fleet-level balance.

The same trade-off is visible within the PPO family. Compared with PPO-MLP-63D, Attn-PPO substantially reduces total completion time and queue variance, but its average

TABLE I
LEARNING HYPERPARAMETERS USED IN THE COMPARISON

Method	Network / Encoder	Learning Rate	Rollout / Update	Batch	Discount	Other Settings
PPO-MLP-63D	MLP actor/value, [128,128], ReLU	3×10^{-4}	$n_{\text{steps}} = 2048$	64	$\gamma = 0.95$	$\lambda = 0.95$, $\epsilon = 0.2$, 10 epochs, entropy 0.01
Attn-PPO	12-token attention, 64-D, 4 heads, 128-D projection	3×10^{-4}	$n_{\text{steps}} = 2048$	64	$\gamma = 0.95$	same PPO settings as PPO-MLP-63D
A2C	MLP actor/value, [128,128], ReLU	3×10^{-4}	$n_{\text{steps}} = 16$	–	$\gamma = 0.95$	$\lambda = 0.95$, entropy 0.01, RMSProp $\epsilon = 10^{-5}$
DQN	MLP Q-network, [128,128], ReLU	1×10^{-4}	train freq. 4, target update 1000	64	$\gamma = 0.95$	buffer 10^5, learning starts 1000, ϵ-greedy 1.0 to 0.05 over 10% training

TABLE II
PERFORMANCE COMPARISON ACROSS DISPATCHING ALGORITHM FAMILIES

Algorithm / Family	Total Completion Time (s)	Average Movement Distance (m)	Average Waiting Time (s)	Queue Variance
Attn-PPO	633.14±12.68	189.14±11.14	357.26±30.12	**0.76±0.65**
PPO-MLP-63D	753.64±8.71	180.23±9.26	327.93±45.41	2.22±2.07
PPO-MLP-54D	1502.36±536.92	303.43±85.33	564.61±111.05	3.02±4.05
A2C	759.90±12.35	183.44±11.50	313.64±28.97	1.16±0.53
DQN	1615.54±834.79	299.36±150.46	353.56±131.21	6.49±6.87
Greedy	951.82±121.27	177.34±5.47	**291.16±0.19**	1.56±0.00
Round-Robin	1193.40±145.15	441.99±52.47	603.66±50.95	3.82±2.55

TABLE III
INTERNAL PPO VARIANCE EVIDENCE

Variant	Completion Std.	Queue-Var. Std.	Evidence Status
PPO-MLP-54D	536.92	4.05	RMF repeated runs
PPO-MLP-63D	8.71	2.07	RMF repeated runs
Attn-PPO	12.68	0.65	RMF repeated runs

waiting time is higher. This pattern is consistent with the reward in Eq. (2): the queue penalty $c_q q_t^2$, the makespan-improvement reward $c_m \Delta M_t$, and the completion rewards $c_c n_t^{\text{done}}$ and $c_a \mathbf{1}_{\text{all}}$ all favor decisions that shorten the fleet-level schedule and avoid overloading a single robot. The waiting-related terms still penalize idle waiting and delayed service, so Attn-PPO is not trained to ignore waiting time. Rather, the attention encoder appears to help PPO exploit the same reward by representing robot–task–queue interactions more directly, leading it to implicitly prioritize makespan reduction and workload balance when those objectives conflict with lower average waiting time.

B. Estimated-Time Feature Ablation

The PPO-MLP-54D ablation removes the 3×3 estimated start-time guidance block. This change makes the MLP policy less stable in repeated validation. The reason is that the flat MLP no longer receives explicit guidance about how soon each of the three UGVs can start each of the first three visible pending tasks. It must infer this relation indirectly from robot positions, queue lengths, task locations, and global statistics. Small changes in task order, queue state, or training randomness can therefore lead to different assignment patterns and higher variance.

This ablation suggests that the estimated-time features are not merely redundant extra inputs. For the flat PPO-MLP policy, they act as heuristic guidance features that stabilize robot selection. A complete multi-seed study should report PPO-MLP-54D, PPO-MLP-63D, and Attn-PPO learning curves with shaded variance under matched training seeds.

C. Internal PPO Variance Evidence: MLP vs. Attention

All PPO modes use the same policy-gradient objective, so the main difference lies in the representation learned before the actor and critic. PPO-MLP-63D uses a fixed fully connected encoder, which treats the observation as a flat vector. Attn-PPO introduces an inductive bias that better matches the dispatching problem: robots, tasks, and global state are represented as interacting tokens.

The standard deviations in Table III are computed from the final summary rows of five RMF evaluation logs for each PPO variant. They therefore measure the spread across the available RMF runs for the selected trained policy configuration of each variant, including shuffled task order and simulator execution effects. The logs do not encode independent training-seed identifiers, so these values should not be interpreted as a clean decomposition of rollout variance versus training-checkpoint variance, nor as a complete multi-seed retraining estimate. If a policy checkpoint was replaced between logged RMF runs, that checkpoint variation is also folded into the reported standard deviation.

This limitation is especially important for PPO-MLP-54D, whose large completion-time standard deviation in Table II indicates poor observed reliability but does not by itself identify the cause. The variance could arise from unstable learning, sensitivity to initialization, different selected checkpoints, or interaction with shuffled RMF task orders. A stronger training-stability comparison would retrain PPO-MLP-63D and Attn-PPO under at least three to five matched random seeds, plot episodic return versus environment steps with shaded standard-deviation or confidence bands, and report the final validation return and RMF completion metrics as mean $\pm$ standard deviation across independently trained policies. Such seed-controlled curves are not included in the present evaluation, so the robustness claim here is limited to the available RMF rollout evidence rather than training stability across seeds.

The observed workload-balance improvement is consistent with this structure. A robot token can attend to pending task

tokens and global congestion information before the final action is selected. As a result, the policy can model robot–task compatibility, queue pressure, and active-task availability more directly. The available quantitative evidence is strongest for queue balance in this single airport-terminal configuration: Attn-PPO reduces the standard deviation of queue variance from 2.07 for PPO-MLP-63D to 0.65 in Table III. The completion-time standard deviation is not lower for Attn-PPO, so the term "more stable" should be understood specifically as more stable workload balancing in the reported RMF runs, not as uniformly lower variance on every metric or as a demonstrated scalability result.

The present analysis does not yet identify which attention links or token features are most responsible for this behavior. A more direct analysis would visualize attention weights from representative dispatch states, such as a heavily imbalanced queue state and a balanced queue state, to check whether robot tokens focus on congested robots, high-waiting-time tasks, or the global queue statistics. A complementary token-design ablation would remove the estimated start-time vector $e_{t,j}$ from the task token in Eq. (3), remove global queue-statistic features, or pool only robot tokens without the global token. Such experiments would separate the effect of generic self-attention from the effect of the specific RMF observation decomposition used here. These interpretability and token-ablation results are not included in the current paper, so the explanation for Attn-PPO remains a plausible mechanism supported by Table III, not a fully isolated causal attribution.

D. Trade-Off Between Local Efficiency and Fleet-Level Scheduling

Attn-PPO and greedy scheduling occupy different positions in the metric trade-off. Greedy is attractive when the operator prioritizes immediate waiting time or travel distance in this fixed map. Attn-PPO is preferable in the tested three-UGV scenario when the operator prioritizes mission completion time and balanced UGV workload. Without an externally specified weighted utility function, the results should be interpreted as a Pareto-style comparison rather than as a single absolute ranking. If a scalar deployment objective is required, the four metrics in Table II should be normalized and weighted according to operational priorities before selecting a dispatcher.

VI. Conclusion

This paper presented a PPO-based UGV task dispatching framework for a UAV-assisted Open-RMF airport terminal simulation. The dispatcher uses a 63-dimensional observation and an RMF-compatible four-action interface, and it is compared with A2C, DQN, greedy scheduling, and round-robin scheduling under the same task pipeline.

The results show that Attn-PPO achieves the best total completion time and queue balance in the tested configuration, while greedy scheduling remains stronger on average waiting time and movement distance. Within the PPO family, PPO-MLP-63D serves as a strong flat-vector baseline because the 3×3 estimated start-time guidance block provides useful heuristic information. The available ablation evidence suggests that removing this block makes PPO-MLP more sensitive, but complete multi-seed quantitative results for PPO-MLP-54D are still needed. In the reported single-map setting, the attention-enhanced PPO mode improves workload-balance stability by explicitly modeling interactions among robots, tasks, and global queue state.

The scope of the conclusion is limited to the reported airport-terminal map, three dispatchable UGVs, one task arrival every 20 s, and at most eight visible pending tasks. The present results do not establish qualitative robustness to different waypoint layouts, four or more UGVs, different arrival rates, or larger visible queues. Future work will evaluate larger and more dynamic task sets, alternative maps, different fleet sizes, varied arrival processes, expanded task queues, and complete learning curves under independent seeds. It should also add attention-weight visualizations and token-design ablations, including removal of estimated start-time features from task tokens, to determine which parts of the structured observation are most important. A major extension is to make UAVs dispatchable by adding UAV tokens, battery state, flight corridors, payload constraints, sensing status, and heterogeneous UAV–UGV action semantics. Physical robot validation is also required before deployment claims can be made.

References

[1] B. P. Gerkey and M. J. Mataric, "A formal analysis and taxonomy of task allocation in multi-robot systems," *The International Journal of Robotics Research*, vol. 23, no. 9, pp. 939–954, 2004.

[2] H. Chakraa, F. Guérin, E. Leclercq, and D. Lefebvre, "Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art," *Robotics and Autonomous Systems*, vol. 168, Art. no. 104492, 2023.

[3] H. Aziz, H. Chan, Á. Cseh, B. Li, F. Ramezani, and C. Wang, "Multi-robot task allocation—complexity and approximation," in *Proc. International Conference on Autonomous Agents and Multiagent Systems*, 2021, pp. 133–141.

[4] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[5] Z. Peng, J. Zhu, L. Li, X. Liu, H. Jiang, and J. Li, "Dynamic collaborative scheduling optimization of multi-robot based on PPO in air-ground collaboration scenes," 2026, doi: 10.1109/RPIC68328.2026.11518051.

[6] L. C. D. Bezerra, A. M. G. dos Santos, and S. Park, "Learning policies for dynamic coalition formation in multi-robot task allocation," *IEEE Robotics and Automation Letters*, 2025.

[7] X. Lin and R. Tron, "Adaptive bi-level multi-robot task allocation and learning under uncertainty with temporal logic constraints," in *Proc. International Conference on Autonomous Agents and Multiagent Systems*, 2025, pp. 1336–1344.

[8] W. Wang, P. He, B. Wan, Y. Chen, and Y. Zhang, "Learning to reallocate: MAPPO-based spectrum and power optimization for UAV–UGV clusters with dynamic reconfiguration," *Computer Communications*, vol. 249, Art. no. 108434, 2026.

[9] S. Ma, J. Ruan, Y. Du, R. Bucknall, and Y. Liu, "An end-to-end deep reinforcement learning based modular task allocation framework for autonomous mobile systems," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 1519–1533, 2024.

[10] L. Ratnabala, A. Fedoseev, R. Peter, and D. Tsetserukou, "MAGNNET: Multi-agent graph neural network-based efficient task allocation for autonomous vehicles with deep reinforcement learning," in *Proc. IEEE Intelligent Vehicles Symposium*, 2025, pp. 970–975.

[11] T. G. Varghese, S. Kochuvila, N. Kumar, and R. R. V. Prasad, "Hybrid coordination framework for centralized task allocation and execution in heterogeneous multi-robot systems," *IEEE Access*, 2026, doi: 10.1109/ACCESS.2026.3680629.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[13] M. S. Mondal, S. Ramasamy, and P. A. Bhounsule, "OptiRoute: A heuristic-assisted deep reinforcement learning framework for UAV-UGV collaborative route planning," arXiv:2309.09942, 2023.

[14] M. S. Mondal, S. Ramasamy, J. D. Humann, J. M. Dotterweich, J.-P. F. Reddinger, M. A. Childers, and P. A. Bhounsule, "An attention-aware deep reinforcement learning framework for UAV-UGV collaborative route planning," in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024, pp. 13687–13694.

[15] M. S. Mondal, S. Ramasamy, and P. A. Bhounsule, "How to coordinate UAVs and UGVs for efficient mission planning? Optimizing energy-constrained cooperative routing with a DRL framework," in *Proc. Robotics: Science and Systems*, 2025.

[16] M. S. Mondal, S. Ramasamy, J. D. Humann, J.-P. F. Reddinger, J. M. Dotterweich, M. A. Childers, and P. A. Bhounsule, "Cooperative multi-agent planning framework for fuel constrained UAV-UGV routing problem," arXiv:2309.03397, 2023.

[17] A. B. Asghar, G. Shi, N. Karapetyan, J. Humann, J.-P. F. Reddinger, J. M. Dotterweich, and P. Tokekar, "Risk-aware resource allocation for multiple UAVs-UGVs recharging rendezvous," arXiv:2209.06308, 2022.

[18] M. Ramezani and M. A. Amiri Atashgah, "CFR-MARL: Centralized feedback-driven reward multi-agent reinforcement learning for decentralized cooperative path planning of heterogeneous agents," *Acta Astronautica*, 2026.

[19] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[20] Open Robotics, "Open-RMF: Open Robotics Middleware Framework," 2024. [Online]. Available: https://www.open-rmf.org/